

Design Pattern Mapping

1. Atomic Conditional Update - Preventing Overselling

Problem:

Thousands of customers may attempt to purchase the last available units at the same time. A separate stock check followed by an update can allow multiple requests to claim the same inventory.

Solution:

Inventory is reserved using an atomic conditional database update:

```
UPDATE inventory SET available_quantity = available_quantity - 1 WHERE  
product_id = ? AND available_quantity >= 1
```

If one row is affected, the reservation succeeds. If no row is affected, the product is considered out of stock.

Where Applied:

Inventory Service and Inventory Repository.

2. Idempotency Pattern - Preventing Duplicate Requests

Problem:

Customers may click Buy multiple times, retry after a network failure, or send the same request more than once.

Solution:

Every purchase operation uses an Idempotency-Key. The key and operation result are stored. If the same key is received again, the previously recorded result is returned instead of creating another transaction.

A unique constraint prevents concurrent requests with the same key from creating duplicate operations.

Where Applied:

API Gateway, Checkout Service, Payment Service, and Order Service.

3. Transactional Outbox- Preventing Lost Events

Problem:

A payment may be successfully saved in the database, but the application could fail before publishing the PaymentSucceeded event. This could leave the customer charged without an order being created.

Solution:

The Payment Service stores the payment update and the corresponding outbox event in the same database transaction. A separate publisher then reads the outbox and publishes the event to the message broker.

Where Applied:

Payment Service, Outbox, and Kafka/RabbitMQ.

4. Saga / Compensating Transaction - Handling Cross-Service Failures

Problem:

The purchase process involves multiple services and cannot be handled safely as one distributed database transaction.

Solution:

SALESTORM follows a Saga-style workflow where each successful operation has a corresponding compensating action.

Typical flow:

Reserve Inventory → Process Payment → Confirm Payment → Create Order

If payment definitively fails:

Payment Failed → Release Reservation

If the reservation expires:

Reservation Expired → Release Inventory

Where Applied:

Checkout, Inventory, Payment, and Order Services.

5. State Pattern - Managing Business Lifecycles

Problem:

Reservations, payments, and orders move through multiple states. Allowing arbitrary state changes can result in invalid business conditions.

Solution:

Each business entity follows an explicit state machine.

For example:

AVAILABLE → RESERVED → PAYMENT_PENDING → CONFIRMED → SOLD

Failure paths include:

PAYMENT_FAILED → RELEASED

and:

EXPIRED → RELEASED

Similar controlled transitions are defined for payment and order lifecycles.

Where Applied:

Reservation, Payment, and Order Services.

6. Circuit Breaker - Protecting Against Payment Gateway Failure

Problem:

If the external payment gateway becomes slow or unavailable, continuously sending requests can consume application resources and increase checkout latency.

Solution:

A Circuit Breaker monitors failures from the payment gateway.

When failures cross a defined threshold, the circuit changes from:

CLOSED → OPEN

Further calls are temporarily stopped. After a recovery period, it changes to:

OPEN → HALF-OPEN

A limited number of requests are then allowed to test the gateway. If the gateway recovers, the circuit returns to:

HALF-OPEN → CLOSED

Where Applied:

Payment Service and External Payment Gateway.

7. Retry with Exponential Backoff and Jitter - Handling Transient Failures

Problem:

Temporary network failures, service interruptions, or broker errors may cause an otherwise valid operation to fail.

Solution:

Transient failures are retried using bounded retries with exponential backoff and jitter. The delay increases between attempts to avoid overwhelming a recovering service.

Retries are used only for operations that are safe to repeat, particularly when idempotency is guaranteed.

Where Applied:

Payment gateway calls, event publishing, and message consumers.

8. Bulkhead - Isolating Dependency Failures

Problem:

A slow or unavailable dependency can consume all available threads, connections, or worker resources and affect unrelated operations.

Solution:

Resources are isolated between critical dependencies using separate connection pools, worker pools, and concurrency limits.

For example, slow payment gateway calls must not consume all resources required for inventory or order processing.

Where Applied:

Checkout Service and Payment Service.

9. Dead Letter Queue - Handling Failed Messages

Problem:

Some events may continue to fail because of invalid data, processing errors, or unexpected conditions. Repeatedly retrying such messages can block normal event processing.

Solution:

After a defined number of unsuccessful attempts, the message is moved to a Dead Letter Queue (DLQ). It can then be inspected, corrected, and replayed through a controlled recovery process.

Where Applied:

Order Service and other asynchronous event consumers.

10. Idempotent Consumer - Handling Duplicate Events

Problem:

The message broker uses at-least-once delivery, so the same event may be delivered more than once.

Solution:

Consumers use a unique business identifier or processed-event record to ensure that the same business operation is applied only once.

For example, the Order Service can enforce uniqueness on the payment reference:

```
orders.payment_id UNIQUE
```

If the same PaymentSucceeded event is received again, the existing payment/order relationship is detected and another order is not created.

Where Applied:

Order Service and asynchronous event consumers.

11. Adapter / Strategy Pattern - Supporting Multiple Payment Providers

Problem:

The Payment Service should not be tightly coupled to one specific payment provider.

Solution:

A common PaymentGateway interface defines the operations required by the Payment Service.

Possible implementations include:

RazorpayAdapter
StripeAdapter
MockGatewayAdapter

The Payment Service interacts with the interface rather than directly with a specific provider.

Where Applied:

Payment Service.

12. Reconciliation / Process Manager - Handling Uncertain Payments

Problem:

A payment request may time out after reaching the payment gateway. The customer may have been charged even though SALESTORM did not receive the final response.

Treating the timeout as a failure could incorrectly release the reservation.

Solution:

The payment is first moved to an UNKNOWN state. A reconciliation worker periodically checks the payment gateway for the final status.

Possible outcomes are:

UNKNOWN → SUCCEEDED → continue order creation

UNKNOWN → FAILED → release reservation

UNKNOWN → UNKNOWN → retry reconciliation later

Where Applied:

Payment Service and Payment Reconciliation Worker.